Actividad con IA generativa · «Consultas SQL con un tutor de IA generativa»

Curso: Inteligencia artificial para la docencia universitaria **Asignatura:** Bases de Datos a Gran Escala (BDGE) · Máster en Big Data (UMU/USC) · curso 2026-27 **Recurso publicado:** dsevilla.github.io/bdge-ia-sql · código en github.com/dsevilla/bdge-ia-sql

1. Tabla de la actividad

Item	Descripción	Respuesta
Asignatura	Nombre de la asignatura donde se implementará la actividad.	Bases de Datos a Gran Escala (BDGE), Máster en Big Data (Universidad de Murcia / Universidad de Santiago de Compostela), curso 2026-27.
Contenido Temático	Tema o unidad de la asignatura que se abordará con la actividad.	SQL analítico sobre datos masivos (tema de consultas y procesamiento declarativo): filtrado y orden, agregación con `GROUP BY`/`HAVING`, reuniones (`JOIN` y `LEFT JOIN`), semirreunión con `NOT EXISTS`, expresiones de tabla comunes (CTE), funciones de ventana (`ROW_NUMBER() OVER`), manejo de fechas y documentos JSON. Motor: DuckDB sobre parquet.
Objetivo de la Actividad	Objetivo específico que se pretende alcanzar con la actividad.	Que el alumnado escriba consultas SQL correctas sobre un conjunto de datos real y, a la vez, aprenda a usar un modelo de lenguaje como **tutor** y no como oráculo: la IA no entrega la solución, ayuda a diagnosticar el error y a repasar el concepto que falta.
Herramienta de IAG Utilizada	Nombre y breve descripción de la herramienta de IAG utilizada.	Modelo de lenguaje accesible por API compatible con la de chat de OpenAI (`POST {endpoint}/chat/completions`), con endpoint, modelo y clave configurables por el propio alumno en la página. En la UMU se ha previsto el servicio institucional Nienna (modelos `modelo-gordo`, `deepseek`, `glm`, …). No hace falta clave si se usa la vía alternativa: la página prepara el prompt y el alumno lo pega en un ChatGPT web.
Descripción de la Actividad	Descripción detallada de la actividad, incluyendo pasos a seguir.	La actividad es una web estática (GitHub Pages, sin servidor propio) que carga en el navegador una base de datos real de Stack Overflow en español (~150 MB en parquet) con DuckDB-Wasm, y propone un itinerario de **diez ejercicios** de dificultad creciente. Pasos: **(1)** abrir la página y esperar a que termine la carga de datos; **(2)** escribir el nombre del alumno; **(3)** resolver cada ejercicio escribiendo SQL en su cuaderno y pulsando «Comprobar»; **(4)** el comprobador compara el resultado con la solución de referencia y devuelve un diagnóstico concreto («te faltan columnas», «mismas filas pero distinto orden», «no coincide a partir de la fila 7»); **(5)** si falla, «Preparar pregunta» construye automáticamente un prompt con el enunciado, el esquema, **su** consulta y el diagnóstico del comprobador, y pide al modelo una pista de 120 palabras como máximo sin dar la consulta; **(6)** el alumno pega la respuesta del modelo en la columna 3, decide si le sirve y vuelve a intentarlo (cada intento queda registrado); **(7)** al acertar puede desplegar «Ver la solución de referencia» y compararla con la suya; **(8)** al terminar, exporta el cuaderno en JSON o Markdown como entrega.
Uso de la IA	Indicar si la IA se utiliza para mejorar la productividad docente, integrarse didácticamente en el aula, o ambas.	**Ambas.** *Productividad docente:* el banco de 41 ejercicios con solución de referencia, el comprobador automático de resultados y el generador de prompts permiten preparar y corregir la práctica sin trabajo manual repetitivo. *Integración didáctica:* el alumnado usa el modelo dentro del bucle de resolución, con reglas explícitas (pistas, no soluciones), y debe valorar críticamente cada respuesta.
Recurso Diseñado	Descripción y/o enlace al recurso diseñado o al ejemplo de aplicación en el aula propuesto.	Web interactiva: dsevilla.github.io/bdge-ia-sql (código y datos: github.com/dsevilla/bdge-ia-sql, datos abiertos en dsevilla/bd2-data). Este mismo documento, con el cuaderno de ejemplo resuelto, se publica en PDF: `doc/respuesta.pdf`.
Metodología Implementada	Estrategia metodológica utilizada para integrar la IAG en la actividad.	Aprendizaje basado en problemas con práctica guiada e iterativa: *intento → diagnóstico automático → diálogo con el tutor IA → nuevo intento*. Itinerario de dificultad creciente, andamiaje decreciente (cada vez menos pistas y ejercicios que exigen combinar conceptos) y evaluación formativa del proceso, no solo del resultado.

Item	Descripción	Respuesta
Evaluación de la Actividad	Método de evaluación de la actividad y de los aprendizajes de los estudiantes.	Mixta. **Automática:** el comprobador certifica la corrección de cada consulta comparando el conjunto de resultados (número de columnas, número de filas y contenido/orden) con la solución de referencia; el ejercicio solo se cierra cuando coincide. **Del proceso (rúbrica, ver §3):** calidad de los intentos, uso del tutor (prompts que incorporan el diagnóstico, sin pedir la solución completa) y reflexión final sobre un error propio y una respuesta de la IA descartada. Evidencia: el cuaderno exportado (JSON/Markdown), que registra cada intento con su consulta, el veredicto del comprobador, el prompt enviado y la respuesta del modelo.

2. El recurso diseñado en detalle

2.1 Arquitectura: todo ocurre en el navegador

No hay servidor de aplicación ni base de datos central: la página descarga los parquet publicados en `bd2-data` y ejecuta SQL con **DuckDB-Wasm** dentro de la pestaña. Los ficheros son `Posts1.parquet` (78 MB), `Comments.parquet` (46 MB), `Users.parquet` (15,8 MB), `Votes.parquet` (5 MB) y `Tags.parquet` (50 KB). Ventajas para el aula: no hay que instalar nada, no hay límite de usuarios concurrentes ni coste de infraestructura, y los datos del alumno (nombre, consultas, respuestas del modelo) se quedan en su navegador salvo el texto del prompt que él mismo envía al modelo.

2.2 El comprobador: qué compara exactamente

El comprobador **no compara el texto de la consulta** (hay muchas formas válidas de escribir lo mismo), sino el resultado: número de columnas, número de filas y, celda a celda, el contenido en el orden pedido. De ahí salen tres veredictos posibles:

1. **Coincide** — mismo número de columnas, mismas filas y mismo orden. El ejercicio se cierra.
2. **Mismas filas, distinto orden** — aviso: falta el `ORDER BY` que pide el enunciado.
3. **No coincide** — dice en qué punto: columnas de más o de menos, filas de más o de menos, o la

primera fila donde empiezan a diferir los resultados.

Ese diagnóstico es la materia prima del prompt: la IA no recibe «esto está mal», recibe el mensaje exacto.

2.3 El tutor: qué se le dice al modelo

La página construye el prompt con cinco bloques —enunciado, esquema disponible, la consulta del alumno, lo que dice el comprobador y lo que necesita— y termina con la instrucción clave:

```
Dime en qué me he equivocado, qué concepto de SQL debo repasar y una pregunta que me ayude
a corregirlo. No me des la consulta completa ni la solución: quiero llegar yo. Máximo 120 palabras.
```

Además, el mensaje de sistema del tutor insiste en lo mismo, de modo que el modelo está configurado para dar pistas y no soluciones. La temperatura se fija en 0,2 para que las respuestas sean estables y reproducibles. En el modo alternativo sin clave, ese mismo prompt se copia al portapapeles o se abre en ChatGPT web: la actividad funciona igual, solo que la respuesta se pega a mano.

2.4 Límites y decisiones que conviene conocer

• **La IA puede equivocarse y el alumno debe comprobarlo**: el único árbitro es el comprobador de

resultados. La actividad se diseña precisamente para que la respuesta del modelo se verifique.

• **La clave de API es del alumno** y, si lo pide, se recuerda solo en el `localStorage` de su navegador.

• **El primer acceso tarda** (30-60 s): hay que dejar que termine la carga de los parquet antes de

lanzar consultas.

• **El banco tiene 41 ejercicios**; el itinerario usa 10. El resto queda para ampliaciones, repesca o trabajo autónomo.

3. Rúbrica de evaluación

Criterio	Insuficiente (0)	Adecuado (1)	Excelente (2)
Corrección de las consultas	Cierra menos de la mitad de los diez ejercicios.	Cierra al menos siete, con ayuda del tutor.	Cierra los diez, y alguna solución es además más legible o más eficiente que la de referencia.
Uso del tutor IA	Pide la solución completa o copia la respuesta del modelo sin probarla.	Adjunta el diagnóstico del comprobador en sus prompts y prueba lo que le sugieren.	Formula preguntas cada vez más precisas, contrasta la pista con la documentación y descarta lo que no encaja.
Registro del proceso	Cuaderno vacío o con un único intento por ejercicio.	Registra intentos con su veredicto, prompt y respuesta del modelo.	Además documenta por qué cambió de estrategia entre intentos.
Reflexión final	No identifica ningún error propio ni ninguna respuesta mejorable de la IA.	Identifica un error propio y una respuesta de la IA que descartó.	Explica el concepto que aprendió y cómo lo comprobó con los datos.

4. Anexo I · Los diez ejercicios del itinerario, con su consulta de cierre

Cada bloque recoge el enunciado tal y como lo plantea la página y la consulta final que cierra el ejercicio (la solución de referencia del banco). Es el cuaderno de un alumno que acierta a la primera: por eso la columna «Pregunta a la IA» aparece vacía; en una entrega real recogería el prompt y la respuesta del modelo.

4.1 · Explora las tablas

Enunciado. Muestra en orden alfabético las tablas y vistas de la base. En DuckDB puedes consultar information_schema.tables; limita la salida al esquema main.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT table_type AS Tipo, table_name AS Nombre
FROM information_schema.tables
WHERE table_schema = 'main'
ORDER BY table_type, table_name;
```

4.2 · Ordena preguntas por número de respuestas

Enunciado. Obtén las diez preguntas con mayor AnswerCount. Muestra Id, Title y AnswerCount; desempata por Id ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT Id, Title, AnswerCount
FROM Posts
WHERE PostTypeId = 1
ORDER BY AnswerCount DESC, Id ASC
LIMIT 10;
```

4.3 · Resume las publicaciones por tipo

Enunciado. Para cada PostTypeId calcula cuántas publicaciones hay y su puntuación media. Conserva los grupos con al menos diez filas.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT PostTypeId,
       COUNT(*) AS NumeroPublicaciones,
       ROUND(AVG(Score), 2) AS PuntuacionMedia
FROM Posts
GROUP BY PostTypeId
HAVING COUNT(*) >= 10
ORDER BY PostTypeId;
```

4.4 · Relaciona preguntas y autores

Enunciado. Muestra diez preguntas con su puntuación y el DisplayName del autor. Conserva también las preguntas cuyo autor no tenga una fila referenciable en Users.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT p.Id, p.Title, p.Score, u.DisplayName
FROM Posts AS p
LEFT JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1
ORDER BY p.Score DESC, p.Id ASC
LIMIT 10;
```

4.5 · Encuentra preguntas sin respuesta

Enunciado. Devuelve diez preguntas sin ninguna respuesta. Usa NOT EXISTS y ordena por fecha más reciente e Id para que el resultado sea determinista.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT q.Id, q.Title, q.CreationDate
FROM Posts AS q
WHERE q.PostTypeId = 1
  AND NOT EXISTS (
    SELECT 1
    FROM Posts AS a
    WHERE a.PostTypeId = 2
      AND a.ParentId = q.Id
  )
ORDER BY q.CreationDate DESC, q.Id DESC
LIMIT 10;
```

4.6 · Filtra un mes sin aplicar funciones a la columna

Enunciado. Cuenta las preguntas creadas en marzo de 2021 y calcula su puntuación media redondeada a dos decimales. Filtra con un rango semiabierto (CreationDate >= 1 de marzo y < 1 de abril) en lugar de aplicar month() o year() a la columna: así el filtro puede aprovechar un índice o las estadísticas del fichero.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT COUNT(*) AS Preguntas,
       ROUND(AVG(Score), 2) AS PuntuacionMedia
FROM Posts
WHERE PostTypeId = 1
  AND CreationDate >= TIMESTAMP '2021-03-01'
  AND CreationDate < TIMESTAMP '2021-04-01';
```

4.7 · Resume las preguntas por mes con una CTE

Enunciado. Usa una CTE para contar preguntas por mes y calcular su puntuación media. Muestra los 24 meses más recientes.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
WITH PreguntasPorMes AS (
  SELECT strftime(CAST(CreationDate AS TIMESTAMP), '%Y-%m') AS Mes,
         COUNT(*) AS NumeroPreguntas,
         ROUND(AVG(Score), 2) AS PuntuacionMedia
  FROM Posts
  WHERE PostTypeId = 1
    AND CreationDate IS NOT NULL
  GROUP BY strftime(CAST(CreationDate AS TIMESTAMP), '%Y-%m')
)
SELECT Mes, NumeroPreguntas, PuntuacionMedia
FROM PreguntasPorMes
ORDER BY Mes DESC
LIMIT 24;
```

4.8 · Encuentra las tres preguntas mejor puntuadas de cada año

Enunciado. Usa ROW_NUMBER() con PARTITION BY año y filtra después las tres primeras de cada año. Desempata por Id ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
WITH PreguntasPosicionadas AS (
  SELECT Id,
         Title,
         strftime(CAST(CreationDate AS TIMESTAMP), '%Y') AS Anio,
         Score,
         ROW_NUMBER() OVER (
           PARTITION BY strftime(CAST(CreationDate AS TIMESTAMP), '%Y')
           ORDER BY Score DESC, Id ASC
         ) AS Posicion
  FROM Posts
  WHERE PostTypeId = 1
    AND Title IS NOT NULL
    AND CreationDate IS NOT NULL
)
SELECT Anio, Posicion, Id, Title, Score
FROM PreguntasPosicionadas
WHERE Posicion <= 3
ORDER BY Anio DESC, Posicion ASC, Id ASC;
```

4.9 · Construye y consulta un objeto JSON

Enunciado. Construye un objeto JSON con id, título, puntuación y autor para cinco preguntas. Extrae la puntuación del documento con json_extract. Es una adaptación DuckDB del trabajo con JSON de la sesión.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT Id,
       json_object(
         'id', Id,
         'title', Title,
         'score', Score,
         'ownerUserId', OwnerUserId
       ) AS Documento,
       json_extract(
         json_object('score', Score),
         '$.score'
       ) AS PuntuacionExtraida
FROM Posts
WHERE PostTypeId = 1
ORDER BY Id ASC
LIMIT 5;
```

4.10 · Une cuatro tablas: pregunta, respuesta aceptada y los dos autores

Enunciado. Para las preguntas cuya respuesta aceptada la escribió otro usuario, muestra el Id y el Title de la pregunta, el DisplayName de su autor, el Id de la respuesta aceptada, el DisplayName de quien la escribió y la puntuación de la respuesta. Necesitas Posts dos veces (pregunta y respuesta) y Users dos veces (cada autor). Muestra las diez con la respuesta de mayor puntuación; desempata por Id de pregunta ascendente.

Intento	Veredicto del comprobador	Pregunta a la IA
1	Coincide con la solución de referencia · ejercicio cerrado	—

Consulta del intento que cierra el ejercicio:

```
SELECT q.Id AS PreguntaId,
       q.Title,
       uq.DisplayName AS AutorPregunta,
       a.Id AS RespuestaId,
       ua.DisplayName AS AutorRespuesta,
       a.Score AS PuntuacionRespuesta
FROM Posts AS q
JOIN Users AS uq ON uq.Id = q.OwnerUserId
JOIN Posts AS a ON a.Id = q.AcceptedAnswerId
JOIN Users AS ua ON ua.Id = a.OwnerUserId
WHERE q.PostTypeId = 1
  AND ua.Id <> uq.Id
ORDER BY a.Score DESC, q.Id ASC
LIMIT 10;
```

5. Anexo II · Cómo se genera este PDF

```
make          # genera respuesta.pdf a partir de respuesta.md
make clean    # borra el PDF
make deps     # instala lo único necesario: reportlab
```

El conversor es `md2pdf.py` (solo requiere `reportlab`): entiende títulos, párrafos, listas, tablas, bloques de código con recuadro y citas. No necesita LaTeX, pandoc ni conexión a red.